The LeRobot Learning Roadmap

From ROS 2 and OpenCV skills to a two-wheeled 6+1 DoF arm that presses a button and picks up a cup.

Follow the numbers. Move on only when you can meet Pass and answer Explain.

Prepare · Build the robot · Teach · Mission · Safety and limits

How LeRobot works

LeRobot learns a task by copying you. You drive the robot, it saves what the robot sensed and what you commanded, and a neural network learns to produce the same commands from the same observations.

Teleoperate → **Record** → **Train** → **Deploy**

Teleoperate	Record	Train	Deploy
You command the robot with a leader arm and gamepad. Nothing is learned yet.	Each attempt becomes an episode: video, state and action at every time step.	A policy learns to predict your actions from each recorded observation.	The policy drives the robot. You count real successes, not training loss.
`lerobot-teleoperate`	`lerobot-record`	`lerobot-train`	`lerobot-rollout`

Improve: take over when it fails. Your corrections become new data. `--strategy.type=dagger`

The five parts

`Robot`	Your hardware as a Python class: `get_observation()` reads it, `send_action()` moves it.
`Teleoperator`	The human as a Python class: `get_action()` returns your command.
`Processor`	Converts and limits data between teleoperator, dataset and robot.
`LeRobotDataset`	Numbers in Parquet, video in MP4, schema and statistics in `meta/`.
`Policy`	A network that turns one observation into a chunk of future actions.

One time step in your dataset

`observation.images.wrist`	video
`observation.images.head`	video
`observation.state`	25 numbers: 7 arm + 2 base + 16 lidar
`action`	9 numbers: 7 arm + x.vel + theta.vel

Predicting a chunk of actions at once keeps motion smooth and limits compounding errors.

The roadmap

From stage 2 to the end, before any motor moves: e-stop in reach, joint and speed limits set, watchdog armed, arm stowed while driving.

0 Set up the machines — 1 week

- **Learn:** Python 3.12 environments, CUDA wheels on ARM, why versions must match.
- **Do:** Install LeRobot v0.6.0 on the Spark and both Orins. Keep ROS 2 outside the environment.
- **On:** Spark · Robot Orin · Station Orin
- **Pass:** **A real CUDA computation runs on all three, with the same LeRobot version.**
- **Explain:** Why can a generic torch wheel fail on Orin?

1 Read the framework — 1 week

- **Learn:** Dataset format, ACT, action chunks, normalization statistics.
- **Do:** Load a public dataset in Python, train ACT on it, and trace one frame from dataset to predicted action.
- **On:** Spark
- **Pass:** **Plot predicted against recorded actions on an episode kept out of training.**
- **Explain:** Why is low training loss not the same as task success?

2 Write the robot driver — 3 weeks

- **Learn:** Robot class, motor bus, calibration, plugin packaging, two-wheel kinematics.
- **Do:** Write `lerobot_robot_mobile_arm`: 6+1 joints, `x.vel`, `theta.vel`, both cameras, calibration, joint limits, hardware e-stop, watchdog.
- **On:** Robot Orin
- **Pass:** **Joints stay within limits. E-stop and watchdog stop it 5 of 5. Ten 1 m squares each close within 10 cm.**
- **Explain:** Why is there no `y.vel` in your action?

3 Add the lidar — 1 week

- **Learn:** Scan timing, masking the robot's body, reducing a scan to 16 sectors.
- **Do:** Add `lidar.s00` to `lidar.s15` to the observation. Slow and stop `x.vel` inside `send_action`, whatever a policy asks.
- **On:** Robot Orin
- **Pass:** **All 16 sectors log at scan rate with the body masked. Driven at a wall 10 times at top speed, it stops at least 15 cm away.**
- **Explain:** What can a single lidar plane never see?

4 Connect teleoperation — 2 weeks

- **Learn:** Teleoperator class, processors, robot host and client over a network.
- **Do:** Combine the powered leader arm and gamepad into one 9-number action. Run the host on Robot Orin.
- **On:** Robot Orin · Station Orin
- **Pass:** **You complete both tasks 9 of 10 times, watching only the camera feeds.**
- **Explain:** If you cannot do it from the sensors alone, why can the policy not?

5 Record clean data — 1 week

- **Learn:** Episodes, replay, the dataset visualizer, what makes a good demonstration.
- **Do:** Fix camera mounts and lighting. Record 10 test episodes and move them to the Spark.
- **On:** Station Orin → Spark
- **Pass:** **Parked-base replay repeats the task. Feature and camera names are frozen.**
- **Explain:** Why does renaming a camera break a trained policy?

6 Press the button — 2 weeks

- **Learn:** Demonstration variety, ACT training, rollout, reading failures from recorded episodes.
- **Do:** Three steps: parked base, small start offsets, approach from 1 m. Start with 50 demos; add more where it fails.
- **On:** Station Orin → Spark → Robot Orin
- **Pass:** **From 1 m: at least 16 of 20 presses, with no tipping.**
- **Explain:** Which failures come from data, and which from hardware?

7 Pick up the cup — 3 weeks

- **Learn:** Grasp variety, friction, ACT against Diffusion Policy on the same trials.
- **Do:** Empty opaque cup, side grasp, same three steps. Add harder cups only after it passes.
- **On:** Station Orin → Spark → Robot Orin
- **Pass:** **At least 14 of 20: lifted 5 cm, upright, held for 3 s.**
- **Explain:** Why compare policies on identical trials?

8 Teach by correcting — 2 weeks

- **Learn:** DAgger, compounding errors, merging datasets, fine-tuning.
- **Do:** Roll out with `--strategy.type=dagger`, take over before failure, and fine-tune on demos plus corrections.
- **On:** Robot Orin · Station Orin → Spark
- **Pass:** **Beats the earlier policy on the same 20 trials, or you can explain why not.**
- **Explain:** Why do corrections teach what demonstrations cannot?

9 One model, two tasks — 2 weeks

- **Learn:** SmolVLA, language prompts, async inference, Real-Time Chunking.
- **Do:** Fine-tune `smolvla_base` on the Spark with one prompt per task. Serve it from Station Orin to the robot.
- **On:** Spark → Station Orin → Robot Orin
- **Pass:** **On 20 trials per task, compared with your two ACT policies. Either result is fine if explained.**
- **Explain:** What does RTC fix, and what does async inference fix?

10 Run the mission — 1 week

- **Learn:** Sequencing policies, success checks with OpenCV, failure analysis.
- **Do:** Write a Python script: approach, press, confirm the light, pick up the cup. Log every run.
- **On:** Robot Orin · Station Orin
- **Pass:** **At least 6 of 10 full missions, failures classified by cause, and a live demo.**
- **Explain:** Where would your next 50 demonstrations go, and why?

After the mission

- Navigate a room: ROS 2 and Nav2 with the lidar.
- Learn on the robot: HIL-SERL for short arm-only tasks.
- Try larger models: Pi0.5 fits in Spark memory, but expect long runs.
- Share it: publish your dataset and model card on the Hub.

Three computers, three jobs



Robot Orin, on the chassis: Motors, cameras, lidar and safety. Runs ACT directly. Budget its power on the battery.

Station Orin, on the bench: Leader arm, gamepad, monitor. Records datasets to an added NVMe drive. Serves large policies.

DGX Spark, in the lab: Trains every policy, one job at a time, and keeps the master datasets. Never in the control loop.

Most stages: record on Station Orin, train on the Spark, run on Robot Orin.

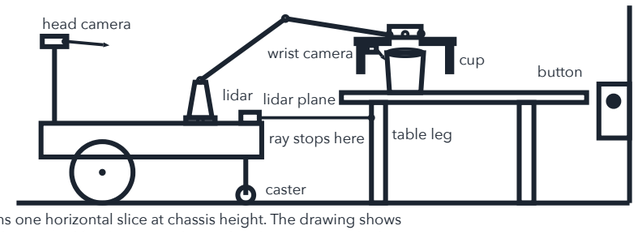
Plan overnight runs: one published SmolVLA example, 20,000 steps at batch 64, took about 30 h on a Spark. Time your own first run.

Spark: `uv venv --python 3.12`, then install from the v0.6.0 tag.

Orin: JetPack 7.2 and Python 3.12. Install the Jetson torch wheel first, then `pip install lerobot==0.6.0 --no-deps` and its dependencies by hand. This recipe was tested on Orin NX, so verify it on AGX.

Both are ARM machines, so LeRobot decodes video with PyAV.

What one lidar plane sees



It scans one horizontal slice at chassis height. The drawing shows the robot parked for the cup; it drives elsewhere to press the button.
Sees: walls, table legs, people's legs, and the button panel when nothing blocks it.
Misses: the cup, the tabletop, the arm, stairs and drop-offs.
So cameras find the targets. The lidar keeps distance and stops the base.
LeRobot has no lidar class. Your driver reports 16 sector distances as floats, and they join observation.state.

How to master it

Run it → **Explain it** → **Change it** → **Debug it**

- **Run it:** The command works.
- **Explain it:** You can say what it wrote and why.
- **Change it:** You alter one part on purpose.
- **Debug it:** You fix a failure from the logs.

- Trace one frame by hand before you trust a command. Follow it from `get_observation()` to the dataset row to the policy output.
- Watch your dataset before you train on it. A bad episode teaches bad behavior.
- Change one thing per experiment, and write it down. Otherwise you cannot tell what helped.
- Count real successes as k of n. 8 of 10 still allows a true rate of 49–94% (95% Wilson interval).
- When the docs and the robot disagree, read the source. LeRobot changes quickly, and the code is the final word.

Times are planning estimates at about 8 hours per week, about 19 weeks in total. Pass targets are starter values: tune them to your hardware and justify the change.

Checked Nov 2026 against LeRobot v0.6 docs. Commands and flags change between releases, so pin one version and use `--help`.